Chapter 1

Introduction

1.1 Project Context

Games have long served as benchmarks for measuring Artificial Intelligence (AI) progress [38]. Many real-world problems, from military tactics to multiplayer games, require intelligent decision-making in environments where all information is not available. A common example is the "fog of war" mechanic found in many multiplayer games [51], where the game environment is not fully revealed to any single player, placing these games in the category of imperfect information games [41]. In such games, players must develop strategies without full knowledge of the game's environmental state [51]. Determining the optimal strategy under these conditions remains a standing challenge for Artificial Intelligence [27].

This study focuses on mastering imperfect information games using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN). In MARL, multiple agents learn and adapt in the same environment, making each decision based on its own observations and rewards [29]. With DQN, the agents use deep neural networks to approximate the Q-function that maps state-action pairs to expected reward. This study introduces the MARQ (Multi-Agent Rainbow Deep Q-Networks) framework, an extension of the DQN architecture with Rainbow enhancements: distributional value estimation (C51) and noisy exploration networks [20]. An AAREN (Attention as Recurrent Neural Network) module [18] processes extended observation sequences and produces learned embeddings for implicit inference of hidden opponent pieces through end-to-end training. League-based self-

play training [35] introduces agents to different historical opponents to prevent similar strategies. Stratego is a good example of a test environment for our framework, agents compete against each other and must develop strategies through repetition of experience while reasoning under uncertainty about the enemy's setup and the environments hidden states.

1.2 Purpose and Description

This research evaluates whether Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) can produce agents that can play imperfect information games at a competitive level. The board game Stratego serves as the test domain because it combines hidden piece identities, a large state space ($\approx 10^{535}$), and long game horizons. Existing analytical tools for Stratego have little to no support for the deeper tactical reasoning that we want to achieve.

The MARQ framework was developed to test whether targeted architectural changes, specifically distributional value estimation, attention-based history encoding, and learned exploration, can overcome the performance ceiling observed in a standard DQN baseline. The experimental results, detailed in later chapters, show that these changes doubled the win rate and reduced the draw rate from 50% to under 7%.

1.2.1 Research Questions

This study is guided by three main research questions:

1. How does the MARQ framework perform in terms of convergence speed and win-rate against heuristic baselines in a partially observable environment?
2. Does the MARQ framework demonstrate statistically significant superiority over standard DQN and rule-based agents across varying opponent difficulty tiers?
3. What is the impact of integrating the AAREN module on the agent's ability to infer hidden opponent piece configurations compared to approaches without explicit belief state modeling?

1.3 Objectives

The main objective of this study is to develop and evaluate the MARQ framework for Stratego, where agents must operate under hidden state variables. The specific objectives are:

- Identify and evaluate reinforcement learning algorithms for competitive play in Stratego.
- Design and implement a game environment for Stratego
- Build and implement a training model using Multi-Agent Reinforcement Learning with Deep Q-Networks and AAREN.
- Train and optimize the AI agent to respond to varying game states, hidden information, and opposing strategies.

1.4 Scope and Limitation

The study is limited to the development of an AI agent for imperfect information games, with Stratego as the test environment. The scope includes the design and implementation of Stratego as a game environment, the development of a training framework using Multi-Agent Reinforcement Learning (MARL) with Deep Q-Networks (DQN) and AAREN, and the training and optimization of the AI agent to adapt to varying game states. Agent performance is evaluated through agent-vs-agent testing and ablation analysis within the Stratego environment. Training is limited to the personal devices of the researchers and does not rely on cloud-based computing resources.

1.5 Significance of the Study

Current imperfect information game systems either require large computational resources or use model-free approaches with limited strategic depth. The MARQ framework contributes a specific combination of techniques: implicit opponent modeling through AAREN-generated embeddings, distributional value estimation via C51, and learned exploration via Noisy Networks. The experimental results show that this combination resolved the scalar value collapse and exploration death observed in a standard DQN baseline, producing an agent that wins 49.88% of self-play games compared to 22% for the baseline. The five-phase curriculum and gradient bridge technique apply to other partially observable or adversarial domains that use recurrent encoders with experience replay.

1.6 Definition of Terms

AAREN (Attention as Recurrent Neural Network)

A neural architecture that combines the parallel training efficiency of Transformers with the sequential processing of RNNs. In the MARQ framework, AAREN processes observation sequences to produce learned embeddings that encode implicit piece identity inference.

Counterfactual Regret Minimization (CFR)

An iterative algorithm that approximates Nash equilibrium by minimizing regret, widely applied in imperfect information games such as Poker and Stratego.

Deep Q-Networks (DQN)

A reinforcement learning algorithm that combines Q-learning with deep neural networks, enabling agents to approximate optimal decision-making in complex environments.

Distributional Reinforcement Learning

An approach that models the full distribution of returns rather than just the expected value. The MARQ framework uses C51 (Categorical DQN) with 51 atoms to capture uncertainty in value estimates.

Fog of War

A game mechanic used in strategy games that conceals parts of the map or opponent actions, requiring players to utilize prediction and information-gathering strategies to reduce uncertainty.

Imperfect Information Games

Games where players have incomplete knowledge of the current game state, requiring them to make strategic decisions under uncertainty (e.g., Stratego, Poker, Multiplayer Online Battle Arenas).

Information Set Monte Carlo Tree Search (IS-MCTS)

A search algorithm for imperfect information games that samples consistent game states from belief

distributions and performs tree search on information sets rather than individual states.

League Training

A self-play training paradigm where agents compete against a diverse pool of historical opponents, preventing strategy cycling and ensuring robust generalization across different playing styles.

Multi-Agent Reinforcement Learning (MARL)

An extension of reinforcement learning in which multiple agents interact within the same environment, learning coordinated or competitive strategies through simultaneous training.

Nash Equilibrium

A game-theoretic concept where no player can gain an advantage by unilaterally changing their strategy, often used as a benchmark for decision-making in imperfect information games.

Perfect Information Games

Games in which all players have complete knowledge of the current game state and available moves, such as Chess and Go.

AAREN Embeddings

Embedding vectors produced by the AAREN module from observed opponent action sequences.

Rainbow DQN

An enhanced DQN architecture that combines multiple improvements including distributional RL, noisy networks for exploration, and dueling network architecture for improved stability and sample efficiency.

Recurrent Neural Network (RNN)

A type of neural network designed to process sequential data by maintaining memory of previous inputs, useful in imperfect information games for recalling past moves or revealed information.

Reinforcement Learning (RL)

A machine learning paradigm where agents learn optimal strategies through trial-and-error interactions with the environment, guided by rewards and penalties.

Stratego

A two-player strategy board game characterized by hidden information and asymmetric piece abilities, where the objective is to capture the opponent's flag or render them immobile.

Chapter 2

Review of Related Literature

2.1 Artificial Intelligence in Imperfect Information Games

Games can be classified as an imperfect information game by having incomplete information about the current game state for both players [38, 29]. An example would be in MOBAs (Multiplayer Online Battle Arena), the fog of war mechanic hides portions of the map, including objectives and enemy positions, from both teams. By placing a tool, such as a ward in League of Legends or an observer in Dota 2, the player gains a temporary vision in the map for strategic planning [13]. This feature engages players to create strategic reasoning under certainty, utilizing prediction, coordination, and resource management to gain informational advantages over opponents. Games have a history of being a metric on how AI progresses and performance in the current time [38, 21]. The case is the same for both perfect information and imperfect information games [38]. For perfect information games, like chess, they primarily use Alpha-Beta pruning, a search algorithm that uses trees to explore possible chess moves and prunes the branches that will not affect the final decision [31]. In imperfect information games, a search algorithm like Alpha-Beta Pruning would be difficult to implement because the algorithm is made for perfect information games [37]. A popular algorithm that is used in imperfect information games would be Monte Carlo Tree Search with Reinforcement Learning because MCTS has integration with neural network systems like AlphaZero [33, 8]. In terms of reinforcement learning, the training focuses on trial and error to provide the best possible decision [54].

2.1.1 Student of Games

A unified learning algorithm that can support both perfect information and imperfect information is Student of Games. Combining self-play learning, strategic search, and principles from game theory [38]. SoG achieved strong performance in Chess and Go in perfect information games and in imperfect information games like heads-up no-limit Texas hold 'em poker, and defeated the state-of-the-art agent in Scotland Yard. When comparing to MARL, SoG supports and has been tested in both game types and uses game theoretic reasoning, which computes or approximates the Nash equilibria. MARL with DQN uses trial and error learning and self-play learning, which relies on the exploration of the environment. MARL has also been tested in complex video games [29], unlike SoG, which has not been tested in video games.

2.1.2 Deep Q-Networks and Dueling Architectures

Deep Q-networks (DQN) and their variants have become a standard framework for decision-making in complex environments with high-dimensional observations. Hessel et al. (2021) combined several independent improvements to DQN, including double Q-learning, prioritized replay, dueling networks, and noisy nets, into a unified agent called Rainbow [20]. Rainbow achieved state-of-the-art performance on competitive benchmarks, and the ablation study showed that each component contributed to the overall gain; removing any single one degraded performance.

A key architectural component of Rainbow is the dueling network architecture, proposed by Wang et al. (2016) and detailed in modern surveys [20, 41]. The dueling design splits the Q-function into a state-value stream and an advantage stream, improving policy evaluation in environments with many similar-valued actions. MARQ uses the same split by feeding its shared ResNet-based representation into dueling value and advantage streams. In the experimental results, this decomposition contributed to the loss-performance coupling observed in the Rainbow agent ($r = -0.787$), where the normal DQN showed no such correlation ($r = 0.051$).

2.1.3 Multi-Agent Reinforcement Learning

Reinforcement Learning can solve complex problems through trial and error, by learning from the environment to be able to provide optimal decisions [29]. This can be the case for Single-Agent Reinforcement Learning(SARL). SARL has its limits in solving many real-world problems. Multi-

Agent Reinforcement Learning(MARL) was introduced as a solution for the challenges of SARL, by having multiple agents in an environment that interact with each other, the decisions they could make would be optimized [29]. Optimized, meaning that these agents go through trial and error together to make the best decision given the reward. As for the implementation of this in Stratego, two agents would compete with each other to know the optimal decisions given the environment, which has imperfect information and opposing strategies. As MARL has its advantages, it also has challenges. The first would be Non-Stationarity, and the second is scalability, as for non-stationarity, each agent learns simultaneously, meaning the environment constantly changes [11]. For scalability, as more agents are being used, the computational power rises, making it expensive [28, 11]. The Ataraxos system [39] addressed non-stationarity through KL-divergence regularization, constraining policy updates toward both exploratory and stable anchors with time-varying coefficients. Ataraxos reached superhuman performance in no-press Diplomacy using this approach. MARQ adopts a similar dynamic damping schedule for its own training stability.

2.1.4 Counterfactual Regret Minimization

Counterfactual Regret Minimization(CFR) is an iterative algorithm that approximates the Nash equilibria commonly used on imperfect information games like poker and Stratego [28, 48]. The Nash equilibria are what CFR approximates to make the best possible decision with minimal regret. Regret in this scenario is the what-ifs of the algorithm if the algorithm chooses the other decision. For CFR to make decisions, it repeatedly minimizes regret to approximate the Nash equilibria [28]. Other works have made variations of the base CFR, showing improved convergence rate, but require a significant amount of effort by researchers. A framework is proposed named AutoCFR, instead of relying on manually crafted algorithms, it automatically designs new CFR variants using an outer loop to search over algorithm designs and an inner loop to test them on equilibrium finding tasks [47].

2.1.5 Opponent Modeling in Imperfect-Information Games

In imperfect-information games, an agent's optimal strategy depends not only on the environment but also on the behavior of the opponent. Classical opponent modeling often builds models from observed action frequencies, and computes best responses in a game-theoretic framework. Li et al. (2024) formalize opponent-model search in games with incomplete information, providing algorithms

to compute optimal or robust strategies given various forms of opponent knowledge [27].

In the deep RL setting, modern approaches encode observations of opponents directly into the network trunk, jointly learning a policy and an opponent model without explicitly predicting actions [41]. This shows that learning implicit opponent representations from interaction traces can be more effective than handcrafted models, especially in high-dimensional state spaces. Our AAREN module follows this tradition of implicit opponent modeling but is specialized to board games with hidden piece identities. Instead of maintaining explicit probability distributions over opponent piece types, AAREN aggregates observed opponent action sequences into dense embedding vectors. These vectors are then concatenated as additional input channels to the Q-network, allowing the agent to reason about the opponent's likely pieces and strategy in a learned representation space.

2.2 RNN

Recurrent Neural Network is a type of Neural Network designed to process sequential data. Unlike common Neural Networks, RNNs have a memory because the information passed to the next step is based on the memory from the previous step. RNN can be applied to imperfect information games such as Stratego because of its memory. By using the memory to remember the pieces that have been revealed, the decision could be optimized [33].

2.2.1 History Aggregation and State Representation Learning

Learning from histories is a central challenge in partially observable and non-Markovian environments. Wang et al. (2026) show how to systematically construct non-Markovian decision processes by aggregating interaction histories into state representations [43]. Their work demonstrates that explicit history modeling through learned aggregators improves agent performance in tasks with complex temporal dependencies.

Representation learning for RL has also explored the use of embeddings to generalize across states and goals. Echchahed and Castro (2025) find in their survey that separating dynamics from rewards enables policy transfer across tasks through learned representations [16]. The AAREN embeddings in MARQ are a form of history-dependent state representation: they encode the opponent's action history into a compact vector that augments the board state. The embeddings are learned jointly with the Q-network rather than handcrafted as belief distributions, and the network infers implicit

piece identities from observed behavior. The experimental results (Chapter 5) show that this joint training produced a 6.5 percentage point accuracy improvement over the LSTM baseline.

2.2.2 LSTM

Long Short-Term Memory is a popular RNN variant that has the capabilities of RNN but also processes data with long-term dependencies [2]. LSTM mitigates the vanishing gradient problem that RNN faces. The vanishing gradient problem is basically as in the name, vanishing gradient, as color in a gradient, the message in this case starts strong and gradually fades away when it passes through each layer, making the learning weak or close to zero. LSTM can be used to solve imperfect information games because of its capability to process data with long-term dependencies while having the memory of RNN.

2.2.3 Aaren

Aaren, known as Attention as a recurrent neural network, combines the parallel training efficiency of Transformers with the streaming update efficiency of RNNs [18]. Aaren, like transformers, can be trained in parallel as well as RNN, can process sequential data with memory. In the study, Aaren was tested in event forecasting, time series forecasting, and classifications. Aaren achieved similar accuracy to Transformers but was more time and memory-efficient.

2.2.4 Residual Networks and Self-Attention in Board Games

Residual networks (ResNets) have become a common backbone in modern board-game RL systems. AlphaZero-style agents for Go, chess, and Stratego use a deep tower of residual convolutional blocks as the network trunk, and this depth is necessary for capturing spatial dependencies on the board [8]. Modern architectures often mix these residual blocks with self-attention layers to bridge local and global knowledge, and the combination has improved playing strength in board games by handling long-range patterns that pure convolutions miss [4].

MARQ adopts a ResNet backbone with spatial self-attention for the same reason: each board position attends to all other positions, capturing long-range piece relationships. The architecture adds a dueling Q-network head and the AAREN opponent embedding on top of this backbone. In the experimental evaluation, the 6-block ResNet with self-attention provided full receptive field

coverage of the 10x10 board while keeping inference latency low enough for real-time play.

2.3 Stratego

The origin of Stratego can be traced to a traditional board game called Jungle, where instead of human pieces instead it used animals. In the jungle, the pieces are not hidden, making it a perfect information game. Stratego is a two-player board game where players each have a 40-piece army. In the tabletop version of Stratego, one player is assigned to the blue army and the other player is assigned to the red army. Each army has a flag assigned to it, which cannot be moved when the game starts. There are a total of 80 troop pieces, while the total number of variations is 12. The pieces rank range from 1 to 10, and a piece that's not numbered is a bomb and the flag. The player also has time to move pieces before the game starts to make use of the fog of war and create creativity and optimal positions on the board. As the pieces have ranks, the higher the number, the higher the power. There is an instance where the lower-ranked can defeat a higher-ranked ranked, which is the piece that has a rank of one, named the spy, can defeat the strongest piece that has a rank of ten, named the marshal, if the spy attacks first. As for the bombs, almost all pieces are vulnerable except the miner, the only one who can defuse the bomb. And the pieces that cannot be moved are the bomb and the flag. Another piece that has a special ability is the scout; it can move either horizontally or vertically, not just one square. If both pieces have the same number, and the piece attacks, both of them would be eliminated. There are multiple win conditions in Stratego. The obvious one would be capturing the flag, another one would be that the opponents have no mobile pieces, another would be that there are no miners that can defuse the bomb to capture the flag, and lastly, the opposing player had the time run out.

2.3.1 Deep Reinforcement Learning in Stratego

Stratego is a popular imperfect-information board game where players must reason about hidden piece identities over long time horizons. Pérolat et al. (2022) demonstrated with DeepNash that a model-free multiagent RL approach can reach human expert level in Stratego, using deep convolutional networks trained with game-theoretic regularization [35]. Their architecture relies on ResNet-style networks to process board features under imperfect information. Subsequent work combined self-play reinforcement learning with test-time search and reached superhuman performance [39].

MARQ uses a similar deep ResNet backbone but adds explicit opponent modeling via AAREN embeddings. The agent conditions its decisions on the opponent's observed action history, and the experimental results show that this conditioning contributed to the win rate doubling from 22% (normal DQN) to 49.88% (Rainbow with AAREN) in self-play.

Figure 2.1: Stratego Game board.

2.4 Other Applications

An extension of MARL with DQN was demonstrated in robotics. In these environments, robots are trained to make decisions based on rewards. The study assigned different energy costs to each robotic arm, allowing the robot to learn how to adjust its movement based on cost, using less energy when the reward is small and exerting more effort if the reward is appropriate [30]. MARL with Deep Q-learning was also used in traffic signal control. Applied a cooperative multi-agent DQN framework to manage six interconnected intersections, where each intersection acted as an independent learning agent. Using the SUMO simulator, each agent selected signal phases to minimize congestion and improve overall traffic flow. Their approach introduced a reward function based on the standard deviation of stopping vehicles across lanes, showing balanced lane utilization rather than simply reducing queue lengths. This design showed that MARL with DQN can enhance not only traffic efficiency but also environmental sustainability by reducing emissions and fuel consumption. In businesses in developing countries, it is essential that growth should be the top priority; however, this creates a problem in increasing taxation for improving infrastructure to further support the current population. Using MARL, we can create a model system where multi-agent RL acts as different parties that discover the optimal tax policies without needing to rely on traditional economic models [53].

2.5 Application to Other Game Environments

Although this study focuses on Stratego, the core MARQ components (distributional Q-learning, attention-based history aggregation, and league self-play) are not Stratego-specific. Other game domains with hidden information could use the same architecture. Sports management games such

as Football Manager involve sequential decisions under uncertainty about player development and market dynamics. Real-time games such as Clash Royale involve hidden opponent hands and resource levels. These domains share the partial observability structure that MARQ was designed for, though applying the framework to them would require domain-specific state representations and reward functions. No experiments were conducted outside Stratego in this study.

Chapter 3

Theoretical Frameworks

3.1 Stratego Game Domain

Stratego is an imperfect information board game with European roots, popularized in the Netherlands in the 1940s and released in North America in 1961 [35]. The game is played on a 10x10 board that includes two 2x2 "lake" areas in the center, which are impassable. Each player commands an army of 40 pieces of varying ranks, and during the game, the identity and rank of the pieces are kept secret from the opposing player. The objective is to capture the opponent's flag or eliminate all of their movable pieces.

Each army consists of 12 different types of pieces, with ranks ranging from 1 (the Spy) to 10 (the Marshal), alongside Bombs and a Flag. Higher-ranked pieces defeat lower-ranked ones during combat, except in special situations: the Spy can defeat a Marshal if attacking first, the Miner is the only piece that can defuse Bombs, and Scouts can move multiple squares in a straight line. There are 3 ways to win the game capturing the flag, immobilizing all movable enemy pieces, or forcing the opponent to run out of time.

Figure 3.1: Stratego pieces.

Figure 3.2: Stratego Game board.

The high theoretical state space of Stratego shows how difficult the game is to solve:

$$|\mathcal{S}| = \binom{40}{1,1,2,3,4,4,4,5,8} \times 10^{10} \times 2^{40} \approx 10^{535} \tag{3.1}$$

Perolat et al. [35] established the permutation for game state. The agent must use reasoning with limited information rather than trying to look at every possible outcome, making the computation intractable.

3.2 MARQ Framework Overview

The MARQ framework is a Multi-Agent Rainbow Deep Q-Networks system that trains agents through self-play and deep reinforcement learning [8]. Agents learn by competing against many different opponents, including their own previous versions and exploratory variants, within a league-based self-play system.

The central problem is reasoning about hidden information without enumerating every possible state. The number of possible game states exceeds 10^{535}, and initial setups reach 10^{66} per player [35]. MARQ handles this scale through implicit belief representations, probabilistic reasoning, and pattern recognition. Deep reinforcement learning estimates values while attention mechanisms process game history and mixed strategies prevent exploitation, all within real-time computational constraints.

The architecture connects these components in a pipeline. The AAREN module takes raw game positions and move sequences and produces learned embeddings that encode opponent piece identity inferences. These embeddings are the input for all subsequent reasoning. A league-based training method trains and evaluates MARQ agents simultaneously.

3.3 MARQ Policy Definition

Policy optimization in reinforcement learning improves an agent's strategy to maximize total reward [41]. Games with hidden information and multiple players require policies that perform well against fixed opponents and also handle uncertainty and shifting strategies from other agents. MARQ combines belief state reasoning with value-based decisions.

Policy learning in MARQ serves two goals. The first is winning games through tactical choices. The second is improving the quality of the embeddings that the AAREN module produces. The

Table 3.1: Key symbols and descriptions.

Symbol	**Description**	**Example (MARQ Framework)**
$\pi_i(a\|I_i)$	Policy for agent i choosing action a given information set I_i	Probability of moving a piece forward given current visible board state and belief about opponent pieces
$\mathcal{B}_t$	Implicit belief representation at time t	Probability distribution over possible identities and positions of hidden opponent pieces inferred by AAREN
$v_i^{\pi}(h, \mathcal{B}_t)$	Value function for agent i following policy π given history h and belief $\mathcal{B}_t$	Expected reward for current board position considering uncertainty about opponent setup
$Q_{\text{network}}(s', a)$	Deep Q-Network approximation of action-value function	Neural network estimate of value for attacking with a suspected Marshal vs opponent piece
$R^{\text{capture}}i, t+1$	Immediate reward from piece captures weighted by piece values	+9 points for capturing opponent Marshal, -1 for losing a Scout
$O = \{o_1, \ldots, o_t\}$	Observation sequence processed by AAREN attention mechanism	Sequence of board states, moves, and attack outcomes throughout the game
α_i	Attention weights for historical observations in AAREN	Higher weight on turn when opponent's Marshal was revealed, lower on routine Scout moves
π^*	Nash equilibrium policy profile	Optimal mixed strategy that cannot be exploited by opponent strategy changes
$Q_{eval}(\mathcal{B})$	Embedding quality evaluator output	Confidence score for learned representation accuracy
$H(\pi)$	Policy entropy measure	Ensures minimum unpredictability to prevent exploitation
ϵ	Exploration rate for agent behavior	Controls exploration vs exploitation balance in multi-agent learning
γ	Discount factor for future rewards	Importance of long-term vs immediate rewards in value estimation

two objectives reinforce each other: better embeddings sharpen decision-making, and good decisions reveal opponent information that in turn refines the embeddings.

3.3.1 Core Policy Framework

A policy in the MARQ framework, denoted $\pi_i(a|I_i)$, defines a probability distribution over actions for agent i conditioned on its current information set I_i. In perfect information games, policies operate on complete game states. MARQ policies operate under uncertainty about the true state of the environment. The information set I_i contains all observations available to the agent: visible board positions, revealed piece identities, and the history of observed movements and combat outcomes.

The core difficulty is that agents must act without knowing the opponent's private information. In Stratego, this means the rank and position of every unrevealed opponent piece remain unknown. The policy needs to take account for variations of the possible hidden pieces alongside the opponents strategy.

MARQ uses a implicit belief representations $\mathcal{B}_t$ rather than calculating every possible game state. The AAREN module learns and updates these representations as new information arrives. The policy takes the following form:

$$\pi_i(a|I_i, \mathcal{B}_t) = P(\text{action} = a \mid \text{information set } I_i, \text{implicit belief } \mathcal{B}_t) \tag{3.2}$$

The policy learns to weight actions based on their expected value across the distribution of opponent configurations rather than overfitting to a single enemy configuration.

3.3.2 Implicit Representation Optimization

MARQ treats representation accuracy as a training objective and uses revealed piece identities as supervised labels for embedding refinement.

When combat reveals the piece identities, the system calculates the prediction error using Kullback-Leibler (KL) divergence.

$$\mathcal{E}_{\text{pred}}(t) = \sum_{p \in \text{Pieces}_{\text{revealed}}(t)} D_{\text{KL}} \left(\text{Representation}_t(p) \| \delta_{s_p^*} \right) \tag{3.3}$$

Here, $\delta_{s_p^*}$ is the Dirac delta distribution concentrated on the true piece type. The error quantifies the distance of the predicted distribution to the actual identity.

This prediction error becomes is the reward signal that encourages the policy to better predict actual piece configurations:

$$R_{\text{accuracy}}(t) = -\mathcal{E}_{\text{pred}}(t) = -\sum_{p \in \text{Pieces}_{\text{revealed}}(t)} \left[\text{Representation}_t(p = s_p^*) > 0\right] \cdot \log \text{Representation}_t(p = s_p^*) \tag{3.4}$$

The optimal policy maximizes:

$$\pi_i^* = \arg\max_{\pi_i} \mathbb{E}_{\tau \sim \pi_i} \left[\sum_{t=0}^{T} \gamma^t \left(R_{\text{game}}(t) + \eta \cdot R_{\text{accuracy}}(t) \right) \right] \tag{3.5}$$

The hyperparameter $\eta \in \mathbb{R}^+$ controls the accuracy versus immediate game rewards. The value function in MARQ extends the standard Bellman equation to account for game state belief, expressing the expected cumulative reward as a function of the current known information set and implicit belief.

$$v_i^\pi(I, \mathcal{B}_t) = \mathbb{E}_\pi \left[R_{i,t+1}^{\text{capture}} + \lambda_1 R_{i,t+1}^{\text{info}} + \lambda_2 R_{i,t+1}^{\text{position}} + \gamma v_i^\pi(I_{t+1}, \mathcal{B}_{t+1}) \mid I_t = I, \mathcal{B}_t \right] \tag{3.6}$$

Three components make up the reward function. The capture reward R^{capture} for capturing a piece. The information reward R^{info} for information revelation. The position reward R^{position} for territorial control.

MARQ training aims to reach a Nash equilibrium. At equilibrium, no player can improve their expected payoff.

$$v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*) \tag{3.7}$$

Reaching Nash equilibria in a large imperfect information game is computationally intractable. MARQ seeks approximate equilibria through self-play training with diverse opponent populations.

3.4 Deep Q-Network Architecture

The Deep Q-Networks in MARQ operates on implicit belief representations instead of a complete game states [41]. The network approximates Q-values over probability distributions of opponent configurations, $\mathcal{B}_t$:

$$Q(s, a) = \mathbb{E}_{s' \sim \mathcal{S}_t \subseteq \mathcal{B}_t}[Q_{network}(s', a)]$$

A standard replay buffer stores experience. Transitions $(\mathcal{B}_t, a_t, r_t, \mathcal{B}_{t+1})$ are sampled uniformly to train the value function.

3.4.1 Rainbow DQN Components

The implementation uses the standards of the Rainbow DQN architecture [20].

Distributional RL (Categorical DQN). The Categorical DQN (C51) models the value distribution directly. It captures the inherent uncertainty of the game by approximating the distribution with 51 atoms spanning the range from V_{min} to V_{max}.

$$Z_\theta(s,a) = \sum_{i=1}^{N} p_i(s,a)\delta_{z_i} \tag{3.8}$$

Training minimizes the Kullback-Leibler divergence between the predicted distribution and the projected target distribution:

$$D_{KL}(\Phi\hat{\mathcal{T}}Z_{\theta'}||Z_\theta) \tag{3.9}$$

where Φ is the projection operator that maps the Bellman update onto the fixed support atoms.

Noisy Networks for Exploration. Noisy Linear layers adds a learned noise into the end of the network layers. This creates an unpredictable strategy without a fixed exploration variable, and the noise parameters are tuned during training for each state.

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \tag{3.10}$$

Dueling Network Architecture. Compared to a single network in deep Q-networks, the dueling architecture branches to 2 streams for state value estimation $V(s)$ and advantage estimation $A(s,a)$:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a';\theta,\alpha) \right) \tag{3.11}$$

This separation improves policy evaluation when many actions carry similar value.

An exploration bonus is added to the Q-values. The bonus is based on the information gathered by garnering territory, and identifying opponent piece identity.

$$Q_{augmented}(s,a) = Q_{base}(s,a) + \lambda \cdot U(a) \tag{3.12}$$

Here $U(a)$ represents the entropy of the learned distribution at the target location. This term encourages the agent to investigate unknown opponent pieces.

3.4.2 Training Stability via Regularization

The recent study of the imperfect information game, Ataraxos reached superhuman performance [39]. Its designers found that training stability matters as much as the algorithm itself. Deep reinforcement learning agents face two recurring problems which is wasting updates on low-information experiences, a waste of sampling and training, and cycling between strategies without settling on a robust one, non-stationary.

Advantage Filtering. Advantage filtering hand picks the most informative transitions to learn from [44]. In reinforcement learning, the temporal-difference (TD) error measures the gap between predicted values and outcomes. A standard replay buffer samples past transitions at random and stores it in the buffer, treating every move equally. Advantage filtering instead samples the buffer and retains only the transitions with the largest TD errors:

$$\mathcal{B}_{filtered} = \mathrm{Top}_{n/k}\left(\mathcal{B}_{sampled}, |\delta_i|\right) \tag{3.13}$$

where $\delta_i = r_i + \gamma^n V(s_i') - V(s_i)$ is the n-step TD error for transition i.

3.5 AAREN and Learned Embedding System

The AAREN module [18] will serve as the memory and history system within MARQ. It produces learned embeddings from observations that encode the inferences from those observations. AAREN handles long sequences, retaining details over hundreds of moves while avoiding the gradient problems that degrade older models such as LSTMs. An attention mechanism lets AAREN look back across the full sequence of moves at once [18].

3.5.1 Recurrent Attention Computation

AAREN implements an attention-based recurrent computation that maintains a state triplet (a_t, c_t, m_t) across the observation sequence. Traditional recurrent neural networks suffer from vanishing gradi-

ents over long sequences. AAREN avoids this entirely by providing a direct access to the full history through attention weights while maintaining $O(1)$ inference complexity per timestep.

Given a sequence of observations $O = \{o_1, \ldots, o_t\}$ where each o_i encapsulates the visible board state and public action outcomes, the AAREN cell first projects each observation into key and value representations:

$$k_t = W_k(o_t), \quad v_t = W_v(o_t) \tag{3.14}$$

The projection matrices W_k and W_v are learned during training. The key k_t determines how relevant the current observation is for belief updates. The value v_t contains the content to be integrated into the belief representation. The attention score s_t is a numerical representation of each step in the sequence relative other steps and to the learned query vector q:

$$s_t = q^T k_t \tag{3.15}$$

The accumulator a_t aggregates value representations across the 1 episode sequence:

$$a_t = a_{t-1} \exp(m_{t-1} - m_t) + v_t \exp(s_t - m_t) \tag{3.16}$$

The term $\exp(m_{t-1} - m_t)$ adjusts the accumulator when a new maximum appears. The term $\exp(s_t - m_t)$ computes the relative attention weight for the current observation.

3.5.2 Embedding Updates

When a battle reveals a piece's true type, the AAREN embeddings is updated with ground truth. The inferred distribution at the revealed position collapses to certainty:

$$\text{Representation}_{pos}[t] = \begin{cases} 1.0 & \text{if } t = g \\ 0.0 & \text{otherwise} \end{cases} \tag{3.17}$$

The system updates the revealed piece count and adjusts remaining beliefs every time a revelation happens. For unrevealed positions $p \neq pos$:

$$\mathcal{B}_p[t] \leftarrow \mathcal{B}_p[t] \cdot \frac{\text{max\_count}[t] - \text{revealed\_count}[t]}{\text{remaining\_count}[t]} \tag{3.18}$$

3.6 Multi-Agent Learning and Strategy Mixing

3.6.1 League-Based Training Architecture

The league has a changing pool of agents so that the model encounters many different strategies [35]. The main agent is the current policy under training. The historical pool stores earlier versions of the main agent, each representing a different stage of development. Opponents are sampled during training according to a fixed probability distribution, to have less bias on the opponents fought.

$$P(\text{opponent}) = \begin{cases} 0.5 & \text{Main Agent (Self-Play)} \\ 0.5 & \text{Uniform(Historical Pool)} \end{cases} \tag{3.19}$$

Preventing the agent from cycling through the same strategies repeatedly. Reaching mastery of the past versions of itself.

3.6.2 Strategy Mixing for Exploitation Resistance

Deterministic strategies are suboptimal in imperfect information games where opponents have varying strategies [35, 42]. An adversary who knows the deterministic policy of the opponent reduces the game to a perfect information setting enabling the perfect strategy for an opponent, eliminating the strategic uncertainty that defines games like Stratego. MARQ counters this at two levels.

At the action selection level, Noisy Linear layers inject learned noise into the policy. The network learns the appropriate degree of randomization for each game state rather than relying on fixed exploration rate such as ϵ-greedy.

At the strategy level, diverse pool of opponent exposure during training achieves a more generalized learning. The league system maintains a population of historical agent checkpoints, each representing a different strategic approach discovered during training.

3.6.3 Reward Structure

The reward function $R(s, a, s')$ guides learning and shaping agent goals in Stratego's long horizon environment [15]:

$$R_{total} = R_{move} + R_{capture} + R_{tactical} + R_{info} \tag{3.20}$$

The move reward R_{move} gives small bonuses for advancing pieces (approximately +0.05) to encourage the agent to prioritize the territory gain and control. The capture reward $R_{capture}$ scales by the rank of the captured piece according to $0.15 \times \frac{\text{Rank}}{11.0}$. The tactical reward $R_{tactical}$ assigns specific bonuses for key strategic captures: +0.5 for Miner vs Bomb defusals and +1.0 for Spy vs Marshal assassinations. The information reward R_{info} provides +0.3 for revealing high-value opponent pieces.

3.7 Game Environment and State Management

The Stratego environment handles board state management, action validation, and observation filtering for agent training and evaluation.

3.7.1 Information Set Management

The environment maintains distinct representations for each information context:

$$\mathcal{I}_{player} = \{s : s \text{ is consistent with player observations}\} \tag{3.21}$$

Each player receives only the subset of game state information available through its observation function.

3.7.2 Temporal State Evolution

Game states evolve according to an update function.

$$s_{t+1} = \text{EnvStep}(s_t, a_t^1, a_t^2, \theta_t) \tag{3.22}$$

Here θ_t represents stochastic elements such as hidden information revelation and time limits.

3.7.3 Valid Action Filtering

The environment enforces game rules through valid action constraints:

$$\mathcal{A}_{valid}(s, player) = \{a : \text{IsLegalMove}(a, s, player) \wedge \text{RespectsGameRules}(a, s)\} \tag{3.23}$$

Only legal moves consistent with Stratego's rules and piece capabilities are available to the agent.

3.8 MARQ Move Selection and Decision-Making

Algorithm 1 outlines the MARQ move selection process and loop. The first four lines establish the information context. AAREN runs over the public observation sequence to produce an updated history embedding $\mathcal{H}_t$, which is then projected through a softmax layer to yield a probability distribution $\mathcal{B}_t$ over the possible identities of each unrevealed opponent piece. The visible board state S and the history embedding $\mathcal{H}_t$ are concatenated into a single 79-channel tensor, and the Rainbow DQN processes that tensor to produce a Q-value distribution over all actions.

The algorithm then diverges based on move type. Non-attack moves take $Q(s, a_i)$ directly as their final score, since those moves carry no contact uncertainty. Attack moves receive an additional correction. The agent queries $\mathcal{B}_t$ at the target square, retrieves the rank-probability vector P, and computes an expected utility V_{exp} by summing over all possible opponent ranks weighted by the outcome of each encounter. The final score blends the network's learned estimate with this hand-computed expectation through the mixing coefficient λ. A high λ trusts the DQN; a low λ defers to the probabilistic expectation. This separation is necessary because the DQN's value estimate for an attack encodes long-run strategic considerations, while V_{exp} captures the immediate piece-trade arithmetic more explicitly.

The final selection step depends on whether the agent is in training mode and whether the temperature T is above zero. During self-play, a Boltzmann distribution over V_{final} governs sampling, which injects controlled stochasticity and prevents the agent from cycling into a fixed attack pattern that opponents can predict and counter. At evaluation, the temperature is set to zero and the agent commits to the greedy arg max over legal actions.

3.9 Generalizability

3.9.1 Mathematical Convergence Properties

The following bounds hold under standard reinforcement learning assumptions.

Incremental Learning Bounds

The algorithm bounds value function estimation error as follows:

$$\|V_t - V^*\|_\infty \leq \frac{C}{\sqrt{t}} + \epsilon_{approx} \tag{3.24}$$

Algorithm 1 MARQ: Decision-making and Move Selection

Input: Public Observations O, Board State S, DQN weights θ, AAREN weights ϕ, Temperature T
Output: Optimal Action a^*

$\mathcal{H}_t \leftarrow \text{UpdateAggregation}(O, \phi)$
$\mathcal{B}_t \leftarrow \text{GetSoftmaxBeliefs}(\mathcal{H}_t)$
$X_t \leftarrow \text{Concatenate}(S, \mathcal{H}_t)$
$Q(s, \cdot) \leftarrow \text{RainbowInference}(X_t, \theta)$
$\mathcal{A} \leftarrow \text{LegalMoves}(S)$
for all $a_i \in \mathcal{A}$ **do**
  if $\text{IsAttack}(a_i, S)$ **then**
    $target\_pos \leftarrow \text{TargetSquare}(a_i)$
    $P \leftarrow \mathcal{B}_t(target\_pos)$
    $V_{exp} \leftarrow \sum_{r \in \mathcal{R}} P(r) \cdot \mathbb{U}(\text{Attacker}, r)$
    $V_{final}(a_i) \leftarrow \lambda Q(s, a_i) + (1 - \lambda) V_{exp}$
  else
    $V_{final}(a_i) \leftarrow Q(s, a_i)$
  end if
end for
if Training/Self-Play and $T > 0$ **then**
  $\pi(a_i) \leftarrow \frac{\exp(V_{final}(a_i)/T)}{\sum_{a \in \mathcal{A}} \exp(V_{final}(a)/T)}$
  return $a^* \sim \pi(\cdot)$
else
  return $a^* \leftarrow \arg\max_{a_i \in \mathcal{A}} V_{final}(a_i)$
end if

where C depends on reward bounds and the discount factor, and ϵ_{approx} represents function approximation error.

Implicit Representation Accuracy Bounds

Representation accuracy improves with additional observations:

$$\mathbb{E}[\text{Accuracy}(\mathcal{B}_t)] \geq 1 - \exp(-\alpha \cdot N_{obs}(t)) \tag{3.25}$$

where $N_{obs}(t)$ is the cumulative number of informative observations and α is a positive constant.

Strategy Convergence in Self-Play

The league training methodology converges to ϵ-Nash equilibria under standard monotonic improvement assumptions:

$$\lim_{T \to \infty} \|\pi_T - \pi^*\| \leq \epsilon \tag{3.26}$$

for some $\epsilon > 0$ depending on the exploration parameters and opponent diversity. The convergence depends on the diversity of the opponent pool and the promotion thresholds that control which agents enter the main pool.

The experimental results in Chapter 5 are consistent with these bounds. The Rainbow agent's loss continued to decrease over 75,000 episodes (Finding 13), and the win rate in self-play reached 49.88% (Finding 7), though the system was not run long enough to test convergence to approximate Nash equilibrium.

3.10 Algorithmic Evolution and Architecture Ablations

The final MARQ architecture emerged from systematic experimentation and the refinement of several reinforcement learning components. Multiple established algorithms were evaluated and replaced because of performance bottlenecks or insufficient strategic depth in the Stratego domain.

3.10.1 Value Optimization and Exploration.

Early iterations used a normal DQN with ϵ-greedy exploration. This approach proved to be a challenge early on for Stratego's sparse rewards and led to a desync of loss to seing model improvement

though winrate and reward accumlated. The implementation was upgraded to a Rainbow DQN architecture incorporating Noisy Networks for a more dynamic approach to exploration. The standard DQN configuration was then replaced with a Dueling Head architecture, which improved value estimation accuracy in states where multiple actions carry similar utility. To mitigate overfitting to deterministic opponent policies during self-play, the system also employs *Boltzmann Exploration* (Temperature Scaling). The opponent's action selection uses a temperature-scaled softmax over Q-values, injecting controlled stochasticity to produce a more robust training curriculum.

3.10.2 History and Memory Representation.

Initial attempts to manage hidden information in Stratego relied on explicit Probabilistic Belief States, a direct representation using another dqn to estimate the piece value. These states introduced prohibitive computational complexity and memory overhead. They required $O(N^2)$ updates that throttled training throughput. The framework moved to an implicit belief system powered by the AAREN architecture. Long Short-Term Memory cells for trajectory embeddings were also replaced by AAREN's attention-based recurrence, which provided more stable gradient flow over long sequences and better handled the non-Markovian nature of piece identity inference.

3.10.3 Architectural Refinements.

The view of the agent and how it sees the enviroment had several iterations, moving from standard deep convolutional stacks to a dedicated ResNet architecture. Testing identified 6 Residual Blocks as the optimal depth. This configuration provides full receptive field coverage of the 10x10 board while maintaining low inference latency. The optimizer was transitioned from standard ADAM to ADAMW to improve weight decay and policy loss stability.

3.10.4 Strategic Initialization and Efficiency.

The piece placement phase was initially modeled using an agentic SetupAgent trained through deep reinforcement learning. This approach introduced exploitability risks and computational overhead. The project transitioned to a non-agentic, fixed Heuristic Setup Agent with randomized flag positioning. The change resolved the exploitability concern and provided substantial computational savings. The agentic setup required approximately 2 seconds per episode; the heuristic method

completes in under 100 milliseconds. This optimization effectively halved the total training compute requirement. The architectural iterations and the theoretical justifications are systematically documented, linking each component's theoretical necessity directly to empirical validation.

Chapter 4

Methodology

This chapter presents the development and evaluation of the MARQ framework, covering the system architecture, training procedure, and the metrics used to compare agent performance against heuristic and self-play baselines.

4.1 Project Planning

This research followed a rapid prototyping method for the design and construction of the MARQ framework. Development ran from September 2025 to February 2026 and covered building the game environment and training the agent. Algorithmic evaluation and ablation testing ran from March to April 2026.

4.2 System Development

System development consists of two components: the game environment and the MARQ framework. The game environment provides the simulation in which agents operate and train. The MARQ framework produces the trained agent whose performance is evaluated against heuristic baselines and through self-play.

Figure 4.1: MARQ framework architecture.

4.2.1 Piece Value Computation

The reward function requires a quantitative assessment of piece value to guide learning. This section presents an analytical method for computing piece values based on probability theory, as a deterministic alternative to simulation-based estimation.

Combat Dominance Score

The Combat Dominance Score (CDS) represents the probability that a given piece type defeats a uniformly random opponent piece. Let $\mathcal{P}$ denote the set of all piece types and n_i the count of piece type i in standard Stratego. The CDS for piece p is defined below.

$$\text{CDS}(p) = \frac{\sum_{i \in \mathcal{P}} \nVdash_{p \succ i} \cdot n_i}{N} \tag{4.1}$$

where $N = \sum_{i \in \mathcal{P}} n_i = 40$ is the total piece count, $\nVdash_{p \succ i}$ is an indicator function equal to 1 if piece p defeats piece i according to Stratego combat rules, and 0 otherwise. The combat rules specify that higher-ranked pieces defeat lower-ranked pieces, with specific exceptions. The Spy defeats the Marshal when attacking, and the Miner defeats Bombs.

Survival Rate

The Survival Rate (SR) measures the expected probability of a piece surviving an attack from a uniformly random attacker. Let $\mathcal{M} \subset \mathcal{P}$ denote the set of movable piece types (excluding Flag and Bomb). The SR for piece p is calculated as follows.

$$\text{SR}(p) = \frac{\sum_{a \in \mathcal{M}} \nVdash_{p \succeq a} \cdot n_a}{\sum_{a \in \mathcal{M}} n_a} \tag{4.2}$$

The indicator function equals 1 if piece p survives an attack from piece a.

Auxiliary Value Components

Certain pieces have capabilities that other than their combat rank. Table 4.2 enumerates these bonuses.

Table 4.1: Computed piece values with combat and survival metrics.

Piece	**Value**	**CDS**	**SR**
Marshal	8.4	0.825	0.939
General	8.0	0.800	0.939
Colonel	7.5	0.750	0.879
Major	6.7	0.675	0.788
Captain	5.7	0.575	0.667
Lieutenant	4.8	0.475	0.545
Miner	4.3	0.400	0.273
Sergeant	3.8	0.375	0.424
Bomb[†]	2.0	—	—
Spy	1.1	0.050	0.000
Scout	1.0	0.050	0.030
Flag[‡]	0.1	—	—

[†]Immovable defensive piece; eliminates any non-Miner attacker.

[‡]Immovable objective piece; capture results in game loss.

Table 4.2: Strategic ability bonuses for special-capability pieces.

Piece	**Bonus**	**Description**
Miner	2.0	Capable of removing Bombs
Spy	1.5	Defeats Marshal when initiating attack
Scout	1.0	Multi-square movement capability
Marshal	0.5	Highest combat rank

4.2.2 AAREN and Game History

The challenge in Stratego is the partial observability. The framework uses the same architectural approach as DeepNash. In deepnash the technique used was a U-Net Convolutional Torso with Residual Blocks and Skip-Connections [35]. The Attention-as-a-Recurrent-Neural-Network (AAREN) module aggregates history and generates embeddings which then is passed to the CNN and Resnet.

Action Feature Extraction. For each step observed opponent action, AAREN extracts a 24-dimensional feature vector encoding movement features, position features, behavioral indicators, and temporal features.

Figure 4.2: MARQ training architecture.

4.3 Model Training

Training the MARQ agent is computationally intensive due to the large state-action space and the need for accurate belief state representations over extended game horizons.

Rainbow DQN Architecture. The MARQ system uses a Rainbow DQN framework from the implementation of Rainbow DQN paper [**?**]. Categorical DQN (C51) handles value distribution estimation. Noisy Networks drive exploration. A Dueling Network architecture decouples state values from action advantages. The noisy layers introduce state-dependent stochasticity, removing the need for manual epsilon-greedy scheduling.

Figure 4.3: Dueling network heads.

Network Structure. The Q-network features a ResNet backbone for spatial feature extraction. An initial convolutional layer feeds into six residual blocks. This configuration provides a sufficient receptive field to evaluate global board state interactions.

Figure 4.4: ResNet backbone.

A spatial self-attention layer follows the ResNet backbone. It captures relationships between pieces across the full board, even at long range. This layer feeds into the dueling heads, allowing the network to weight distant piece relationships in value estimation.

The 79-channel state representation is partitioned into two functional sets, as summarized in Table 4.3.

The board feature channels provide deterministic knowledge of the agent's own piece positions and observed enemy locations. The AAREN embedding channels encode learned representations of hidden enemy piece identities based on observed behavior. These embeddings are dense learned vectors that the network interprets through end-to-end training, not explicit probability distributions. The AAREN embedding channels are populated in all training phases; the agent always receives embeddings from the history aggregator rather than ground truth one-hot encodings. The board

Figure 4.5: 79-channel input representation.

Table 4.3: State representation input channels.

Channel	Category	Description
Board Features (Channels 0–14)		
0–11	Own Pieces	Binary spatial maps indicating positions of each piece type (Flag, Spy, Scout, Miner, Sergeant, Lieutenant, Captain, Major, Colonel, General, Marshal, Bomb)
12	Enemy Mask	Binary map indicating positions of all visible enemy pieces
13	Obstacles	Binary map of lake squares (impassable terrain)
14	Empty Squares	Binary map of unoccupied traversable positions
AAREN Embedding Features (Channels 15–78)		
15–78	Learned Embeddings	64-dimensional learned embedding vectors per board position, produced by the AAREN history aggregator from observed opponent action sequences. These embeddings encode implicit piece identity inference learned end-to-end with the agent.

feature channels carry different levels of enemy information depending on the observability setting. During full observability (Phase 1), the board tensor reveals all enemy piece identities, providing a strong signal in the piece-specific board channels. During partial observability phases, unrevealed enemy pieces appear only as a "hidden piece" presence marker (affecting Channel 12, the enemy mask), and the agent must rely on AAREN embeddings to infer their true identities. An earlier version suffered from a bug where the hidden piece marker was filtered out of Channel 12, blinding the network to the presence of enemy pieces.

Training Hyperparameters. The training configuration specifies a learning rate $\alpha = 3 \times 10^{-5}$ with AdamW optimizer and weight decay $\lambda = 0.01$, and a discount factor $\gamma = 0.995$ to capture long-term strategic consequences. A batch size of 512 transitions and a replay buffer capacity of 150,000 transitions with prioritized sampling are used. Soft target network updates ($\tau = 0.001$) occur every 5,000 steps. Model updates run every 2 steps across 8 parallel lanes with 5-step returns for deeper credit assignment in long-horizon games. The AAREN history aggregator retains up to 50 actions per position in its training buffer to preserve early-game movement patterns across 200 to 500 move games. Turn normalization is aligned with the maximum game length of 1,000 turns.

Horizontal flip augmentation exploits the game's symmetry to improve sample efficiency. Training proceeds for 35,000+ episodes with checkpoints saved every 1,000 episodes.

Computational Optimizations. Mixed-precision inference accelerates training throughput. During action selection, network forward passes operate in FP16 precision via `torch.amp.autocast`, reducing memory bandwidth requirements and leveraging Tensor Core acceleration on NVIDIA GPUs.

Integrating the recurrent AAREN module into the replay pipeline presents a significant bottleneck. Full sequential reconstruction of action histories during replay increases iteration time from 15s to 60s. To maintain high throughput, MARQ stores pre-computed embedding snapshots in the replay buffer. A "Gradient Bridge" preserves differentiability despite the detached snapshots: a differentiable epsilon-scaled zero-vector ($\epsilon \cdot W_{proj}(\mathbf{0})$) is added to the stored snapshots during replay. This preserves the 4x speedup while providing a gradient path from the DQN loss back to AAREN's input projection layer. The resulting value-staleness is mitigated by AAREN's Dual Training Regime, which provides a fresh supervised signal during piece reveals.

Multi-Phase Curriculum. Training follows a multi-phase curriculum. Phase 1 establishes foundational rules under full observability. As win rates stabilize, the curriculum introduces mixed observability, requiring the agent to use AAREN-based inference before full information occlusion is enforced.

Phase 1 runs for 5,000 to 10,000 episodes against a progressively challenging opponent mix: initially random opponents, then basic heuristic agents, and finally a multi-factor heuristic opponent. Phase advancement requires $\geq$95% win rate against random opponents, $\geq$75% against the basic heuristic, and $\geq$60% against the strategic heuristic. All win-rate metrics are computed relative to decisive games (wins and losses) rather than total episodes to account for the high frequency of early-stage draws.

Phase 2 (Partial Observability) disables full observability and requires reliance on AAREN-generated embeddings for 5,000 to 10,000 episodes. The opponent distribution allocates 40% to frozen heuristic agents and 60% to the strategic heuristic. Phase advancement requires embedding-based inference accuracy $\geq$75% and an overall win rate $\geq$60% relative to decisive games.

Phase 3 (Self-Play) trains against a mixture of delayed agent copies (60%) and strategic heuristic opponents (40%) for 5,000 to 15,000 episodes to discover novel strategies while preventing policy

collapse. Phase 4 (League Training) activates the full opponent distribution with 40% historical league agents, 30% strategic heuristic, 20% greedy agents, and 10% self-play. This phase also activates independent parallel-learning (MARL) for Agent 2, transitioning the opponent from a fixed policy pool to a continuously updating student agent. Phase 5 (Scenario Drills) introduces specialized board configurations every 1,000 episodes for targeted skill training.

4.3.1 Test-Time Search

The trained Q-network provides a strong action-selection policy through single-pass inference. Tactical performance in partially observable environments improves with a formal *Expectamax* search, originally proposed by Michie [32] and recently analyzed in complex game environments by Novikov and Yanovskyi [32]. This algorithm extends standard minimax by incorporating stochasticity through "Chance Nodes" that represent the hidden identity of opponent pieces.

The search refinement uses the piece-identity probabilities generated by the AAREN history aggregator to calculate expected utility of potential moves. For a given state s and a set of legal moves $\mathcal{A}(s)$, the refined value $V^*(a_i)$ is computed as follows:

1. **Max Node evaluation.** The agent evaluates each candidate move a_i using its positional prior $Q_{dqn}(s, a_i)$.

2. **Chance Node summation.** If a_i results in an attack on position j, the identity of the target piece is treated as a chance node. The expected combat utility $\mathbb{E}[U_{combat}]$ sums the utility of all possible outcomes, weighted by their probabilities:

$$\mathbb{E}[U_{combat}] = \sum_{r \in \mathcal{R}} P(\text{rank}_j = r|\mathcal{H}) \cdot \text{Utility}(rank_{attacker}, r) \tag{4.3}$$

   where $P(\text{rank}_j = r|\mathcal{H})$ is the AAREN-derived probability that the target piece at position j has rank r, and $\mathcal{H}$ denotes the observed interaction history.

3. **Move Selection.** The final decision combines the positional prior and the expected tactical outcome:

$$a^* = \arg\max_{a_i \in \mathcal{A}(s)} (\lambda \cdot Q_{dqn}(s, a_i) + (1 - \lambda) \cdot \mathbb{E}[U_{combat}]) \tag{4.4}$$

The Expectamax logic prevents the agent from committing high-value pieces to squares with high probabilities of containing Bombs or superior ranks, even when the raw Q-network output is

optimistic about the board position. The exact summation over rank probabilities grounds the risk assessment in observed opponent behavior.

4.3.2 Deployment

The best-performing MARQ model weights are saved after training and validation. During evaluation, exploratory noise is deactivated, and the model selects the action with the highest predicted Q-value. Each evaluation run uses the same checkpoint to maintain consistency across test conditions.

Figure 4.6: Game sequence diagram.

The game loop runs until one agent captures the opposing flag. On each turn, the active agent receives the current board state and updates the AAREN module, which produces learned embeddings from observed action history. The DQN evaluates Q-values for all legal actions given this embedding-enhanced state representation and selects the action with the highest expected value.

4.4 Game Environment

The game environment is a digital implementation of Stratego developed in Python and Pygame. It manages the board state, enforces game rules, handles combat resolution, and provides the observation interface for agent perception. All agent-vs-agent experiments in this study use this environment.

4.4.1 Evaluation

The evaluation tracks learning progress and agent performance through agent-vs-agent testing. Win rate measures the proportion of decisive games won against each opponent tier. Average game length, computed as the mean number of moves per game, indicates decision efficiency. Training stability is assessed through loss curves showing the progression of distributional and value losses over episodes. Ablation comparisons between the Rainbow DQN agent and a standard DQN baseline isolate the contribution of each architectural component. Statistical significance is determined

through paired comparisons across evaluation windows. Empirical findings are organized thematically into designated subsections to clearly connect each architectural modification to its resulting behavioral outcome.

Developing the MARQ framework is iterative. Each version represents an improvement over the last. Appendix B shows the initial results from 1,000 episodes. During this early stage, the agent is still exploring strategies. The early win rates show many draws as pieces move without decisive engagement. With Noisy Networks, the agent learns how much to explore on its own, adapting better as training continues.

Chapter 5

Results and Discussion

5.1 Normal DQN Performance Ceiling

The multi-phase curriculum concluded with 37,213 episodes of self-play training. The normal DQN agent used an LSTM-based history encoder and trained against random, greedy heuristic, league, and self-play opponents. Table 5.1 reports the results.

Table 5.1: Normal DQN performance after 37,213 episodes of self-play training.

Opponent Type	**Episodes**	**Wins**	**Win Rate**	**Draw Rate**
Greedy Heuristic	9,319	1,418	15.2%	41.3%
Self-Play	9,565	2,370	24.8%	36.8%
Random	9,294	2,260	24.3%	37.7%
League	9,035	2,231	24.7%	38.2%
Aggregate	**37,213**	**8,279**	**22.2%**	**38.5%**

The training data contains six findings, each linked to an architectural limitation.

5.1.1 Win Rate Plateau

Agent1's cumulative win rate stayed near 22% after 2,000 episodes, it stayed more or less the same over the next 35,000. The 100-episode rolling average swung between 15% and 30% with no deviation.

Figure 5.1: Cumulative win count and win rate over 37,213 episodes. The win rate plateaus early and remains flat, indicating policy stagnation.

5.1.2 Loss–Performance Decoupling

The TD error fell to roughly 0.001 however, the win rate did not follow. The network learned to predict the value of passive, draw-seeking play accurately. It did not discover better strategies.

Figure 5.2: Policy loss (TD error).

5.1.3 Q-Value Overestimation

Average Q-values climbed from 0.2 to 1.2. Win rates did not move with the rising value estimates.

Figure 5.3: Average Q-values climb.

5.1.4 LSTM Prediction Accuracy Ceiling

The LSTM piece-identity module accuracy stayed around 18–22%, a 2.2–2.6× gain over the 8.3% random baseline but is still far short of reliable tactical inference. The improvement stopped after episode 5,000 and stayed at that level.

Figure 5.4: Piece-identity prediction accuracy.

5.1.5 Opponent-Specific Performance Asymmetry

Performance varies by opponent given randomly in the training process. Win rate against the greedy heuristic was 15.2%; in self-play it reached 24.8%. The policy fit its own behavioral distribution and did not adapt to the others opponent types.

5.1.6 Draw Dominance

The game results was mostly draws. Draws accounted for 38.5% of outcomes, far above the 22.2% win rate. The agent learned to shuffle pieces passively and avoid uncertain engagements. It minimized short-term losses at the expense of long-term winning probability, lacking the long horizon planning.

5.2 Behavioral Patterns from Training Logs

5.2.1 Passive Shuffling Convergence

The agent grew conservative. Average episode length rose throughout training as it avoided combat and cycled pieces between adjacent squares. Scalar value collapse (Finding 2) explains why the Q-network converges to the lowest-variance action class, playing safe moves instead of winning moves.

5.2.2 Opponent-Specific Adaptation Failure

Table 5.1 shows the agent overfitting to its own play style, not generalizing to other agents, since the drop-off of performance against greedy opponents is significant (15.2%). The greedy heuristic has the lowest win rate (15.2%) due to its rank-agnostic attack pattern directly punishing passive shuffling. The heuristic set of instructions attacks the nearest piece, forcing repeated engagements that the agent's risk-averse policy is designed to avoid. When forced into combat, the agent unreliable piece-identity inference needed to choose favorable matchups fails, so each engagement is approximately a coin flip weighted by the opponent's rank distribution. The result is a consistent loss rate that the agent win with further passivity.

Self-play produced the highest win rate (24.8%), but this figure can be misleading. Both agents share the same weights and therefore the same behavioral tendencies. When two equally passive and cautious policies face each other, games resolve through symmetric attrition rather than strategic superiority. The apparent 24.8% win rate reflects the probability that stochastic tiebreakers favor one copy over another, not genuine tactical competence. The high draw rate in self-play (36.8%) confirms that most games end in mutual passivity.

Random opponents produced a 24.3% win rate, nearly identical to self-play despite being the weakest opponent class. A random policy does not exploit the agent's weaknesses, but it also provides no consistent pattern for the agent to learn from. Transitions generated against random play carry high variance and low signal, contributing gradient noise without driving policy improvement. The similar win rates against random and self-play opponents indicate that the agent has not learned to exploit weak play, further evidence of policy stagnation.

League opponents (24.7%) are a historical snapshots of the agent past versions. Because the agent's policy plateaued early (Finding 1), these snapshots are nearly indistinguishable from the current policy. A league of stagnant policies cannot force adaptation and the fixed ϵ-greedy decay

adds to this problem by providing no mechanism for adapting exploration to different opponent since the agent has stopped trying to find different strategy.

5.2.3 Loss-Reward Divergence

The TD loss dropped from 0.05 to 0.001 by episode 37,000. Average episode reward stayed flat near 0.08, a possible confusion in the reward signal. The network learned to predict outcomes of its existing passive play with high precision, knowing that a safe move will net a safe outcome. With the bottoming ϵ-greedy, it did not search for strategies with higher expected returns.

5.2.4 LSTM Accuracy

The piece prediction accuracy hit 20% after episode 5,000 and stagnated for the rest of the training. One correct inference in five is not enough to evaluate combat outcomes reliably, so the agent tend to avoid combat altogether

5.3 Draw Dominance

Draws reached 38.5% of all games, and the win rate was only 22.2%. This rate matters not just as a performance number but as a drag on learning itself, a symptom of a bigger problem.

The scalar value collapse (Finding 2) made the model play safe rather than optimal. A scalar Q-value cannot represent multi-modal outcome distributions. A move with 40% win probability and 60% loss probability (expected value ≈ -0.2) looks worse than a certain draw (-0.3) to a mean-based estimator. The draw-producing move gets the higher Q-value even though it is strategically inferior.

5.4 Architectural Comparison: Unified versus Dual-Model Pipeline

A different approach was use from the initial architecture, which was using two separate networks, a RLNN SetupAgent for piece placement and a DQN for movement. That dual-model design was abandoned, due to the computation overhead and signal interference between the two networks.

The current unified pipeline uses a single DQN for all movement and a randomized heuristic for placement. Table 5.2 compares both.

Table 5.2: Dual-model versus unified architectural pipelines.

Aspect	**Dual-Model (Setup + Attack)**	**Unified DQN**
Architecture	Separate C51-based SetupAgent for placement, separate DQN for attack	Single DQN for movement; heuristic setup with randomized flag
Setup Latency	Approximately 2 seconds per episode	Under 100 milliseconds per episode
Training Overhead	Doubled compute requirement (setup and attack training phases)	Halved total training compute
Exploitability	Deterministic setup patterns learned by opponents	Randomized placement resists opponent exploitation
Draw Pattern	Draws due to gradient interference between competing objectives	Draws due to risk-averse policy under uncertainty

Two lessons emerged from this transition. The SetupAgent did not converged on deterministic placements, randomizing setup while lowering the TD-error of the model a mismatch in result. Placing flag in front of the lineup and trapping big pieces behind the bomb. A randomized heuristic eliminated this vulnerability and cut setup latency by 20×.

The dual-model design also suffered from gradient interference. Setup gradients rewarded defensive formations while attack gradients rewarded aggressive piece use. Optimal defense often places pieces in positions bad for attack. Removing the setup phase from the learning loop resolved this conflict.

Both architectures converged on draw-heavy outcomes. In the dual case, gradient interference between objectives prevented coherent learning. In the unified case, scalar value collapse and poor exploration drove the agent into passive shuffling. The draw problem is architectural and rooted in the long horizon of the game, not an unforeseen accident.

5.5 Architectural Bottleneck Analysis

Findings above are not due to the insufficient training or bad hyperparameters. Each one follows from a known structural property of the DQN architecture.

5.5.1 Q-Value Overestimation

Finding 3 is a known property of single-estimator Q-learning. The network selects and evaluates actions with the same weights:

$$y_t = r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a') \tag{5.1}$$

Noisy estimates plus a max operator produce a positive bias, reducing the ability of the agent to learn from its mistake [20]:

$$\mathbb{E}\left[\max_a (Q^*(s,a) + \epsilon_a)\right] \geq \max_a Q^*(s,a) \tag{5.2}$$

The bias grows with noise variance and action-space size $|\mathcal{A}|$. Stratego positions can have over in theory a 220 to 240 legal moves, amplifying the effect. Double DQN splits selection from evaluation across two estimators, provably reducing this bias [20].

5.5.2 Scalar Value Collapse

Finding 2 (loss convergence without performance gain) follows from distributional RL theory. The standard DQN learns a scalar $\mathbb{E}[Z(s,a)]$ and minimizes:

$$\mathcal{L} = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)^2\right] \tag{5.3}$$

In Stratego the return distribution $Z(s,a)$ is multi-modal [20]: wins near $+1$, losses near -1, exchanges in $[-0.5, +0.5]$, draws near 0. A scalar mean collapses all modes into one number that may not match any real outcome. The loss converges when this mean is estimated correctly, even if the policy cannot tell aggressive high-variance moves from passive low-variance ones.

5.5.3 LSTM Information Bottleneck

Finding 4 follows from sequential compression. The LSTM encodes observations $o_1, \ldots, o_T$ through a 64-dimensional hidden state:

$$h_t = \text{LSTM}(o_t, h_{t-1}) \tag{5.4}$$

A Stratego game runs 200 to 500 moves. The hidden state must track belief estimates for 40 opponent pieces across 12 types. A 64-dimensional vector can hold enough information in theory, but every successive nonlinear update risks losing early observations. Information from move k must survive $T - k$ transformations to reach the final representation. Gating helps, but retention still degrades with sequence length [18].

AAREN sidesteps this bottleneck. It computes weighted aggregations directly over the full history with $O(1)$ per-timestep complexity:

$$h_t = \frac{a_t}{c_t} = \frac{\sum_{k=1}^{t} v_k \exp(s_k - m_t)}{\sum_{k=1}^{t} \exp(s_k - m_t)} \tag{5.5}$$

No sequential compression, no information decay. The network can attend to any prior timestep directly.

5.5.4 Exploration Collapse

The normal DQN relies on a hard-coded ϵ-greedy exploration schedule that decays from 1.0 down to 0.01. Once it hits this floor, the schedule stops adapting. From that point on, the agent explores uniformly at a flat 1% rate regardless of the game state or its confidence in the current Q-values.

Stratego has over 10^{535} possible states [35]. A uniform random exploration rate of 1% is a blunt mechanism that will almost never stumble on the complex sequences of moves required to discover optimal strategies. The agent becomes trapped in local optima because its exploration policy cannot recognize which parts of the state space require further investigation. This limitation necessitates a mechanism like Noisy Networks [20], which replaces hard-coded randomness with learnable parameters that concentrate exploration where the value function is most uncertain.

5.5.5 Sparse Reward Propagation

The normal DQN cannot develop long-horizon strategic behavior (Findings 1, 5, 6), a limitation compounded by single-step TD learning. Terminal reward information at step T propagates backward through the Bellman backup at a rate set by the discount factor γ:

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha \left(r_k + \gamma Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k)\right) \tag{5.6}$$

For a decisive game outcome occurring at step T in a game of length 300, a mid-game decision at step 100 receives an effective signal of γ^{200}. With $\gamma = 0.99$, this yields $0.99^{200} \approx 0.134$, meaning that only 13.4% of the terminal reward signal reaches mid-game decisions through single-step bootstrapping. Multi-step returns (n-step TD) [20] propagate terminal signals n times faster by directly incorporating rewards over n consecutive steps:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, a_{t+n}) \tag{5.7}$$

This reduces the effective bootstrap depth and accelerates the propagation of sparse terminal rewards to earlier decision points.

5.6 Empirical Validation: Rainbow DQN with AAREN

The bottleneck analysis above predicted that each failure mode has a specific architectural fix. The following data mining analysis tests those predictions empirically. Three training datasets were analyzed: the same normal DQN at 15,302 episodes, the Rainbow DQN with AAREN at 9,778 episodes of self-play, and a separate extended Rainbow run of 75,047 episodes against heuristic opponents. Table 5.3 presents the head-to-head comparison. The complete training progress charts appear in Appendix B (Figures **??**–**??**).

Table 5.3: Self-play performance comparison between Vanilla DQN with LSTM and Rainbow DQN with AAREN.

Metric	**DQN+LSTM (15k)**	**Rainbow+AAREN (9.5k)**
Total Episodes	15,302	9,778
Total Steps	10,306,433	5,169,892
Win Rate (%)	24.97	49.88
Draw Rate (%)	50.04	6.69
Loss Rate (%)	24.99	43.43
Steps per Win	2,697	1,060
Avg Episode Length	1,345	1,055
Flag Capture Ratio (%)	18.79	8.88
Loss–Win Rate Corr.	0.051	−0.787
Mean Q-Value	0.33	0.70
Model Size (per ckpt)	∼39 MB	∼433 MB

5.6.1 Win Rate

The Rainbow agent achieved a 49.88% win rate at 9,778 episodes, doubling the result from the DQN architecture. The DQN reached 24.97% at 15,302 episodes despite consuming twice the total environment steps. The Rainbow agent trained on roughly half the data and nearly doubled the win rate, which is a surprising result. Section 5.1.1 (Win Rate Plateau) predicted an architectural ceiling of the DQN architecture while the Rainbow architecture broke through it.

5.6.2 Draw Elimination

The normal DQN drew 50.04% of its games, finishing the 1500 moves per agent in an episode. The Rainbow agent drew 6.69% over 1500 moves. It lost more games outright (43.43% versus 24.99%), but it produced definitive outcomes. An agent that draws half of its games has learned to avoid playing passively and conservative in calculating outcomes in the short term. Section 5.1.6 (Draw Dominance) attributed this to scalar value collapse. The C51 distributional head resolved it.

At first glance, a 43.43% loss rate alongside a 6.69% draw rate could suggest that the Rainbow agent is taking excessive risks rather than learning stable strategies. Three observations argue against this reading. First, the training data comes from self-play, where every game produces exactly one winner and one loser (or a draw). A 49.88% win rate and a 43.43% loss rate sum to 93.31% decisive outcomes. In a zero-sum self-play setting, a high decisive-outcome rate means the agent has learned to force resolutions rather than stall. The losses belong to the copy of the agent that happened to be on the weaker side of a particular game, not to a systematically reckless policy. Second, the steps-per-win metric (1,060 versus 2,697 for the normal DQN) indicates that wins are achieved through efficient, targeted play rather than through drawn-out attrition. An agent that takes reckless risks would show longer, more chaotic games, not shorter ones. Third, the loss–performance coupling coefficient ($r = -0.787$, Section 5.6.3) demonstrates that the agent's losses are informative: as the distributional loss decreases, the win rate increases. The losses the agent incurs during training generate gradient signals that drive subsequent improvement, unlike the uninformative draws that dominated the normal DQN's training. The 75k extended run (Section 5.6.7) provides further evidence that this aggressive posture matures into directed strategy: the agent achieved a 99.96% flag capture ratio among wins, indicating that extended training channels initial aggression into purposeful flag pursuit rather than indiscriminate combat.

5.6.3 Loss–Performance Coupling Restored

The correlation between windowed policy loss and win rate was 0.051 for the normal DQN and -0.787 for the Rainbow agent. As the distributional loss decreased, the win rate increased. Section 5.1.2 (Loss–Performance Decoupling) identified that the scalar DQN loss converged without improving play. The Rainbow architecture made the loss curve became a reliable proxy for gameplay improvement.

5.6.4 Sample Efficiency

The Rainbow agent required 1,060 environment steps per win compared to 2,697 for the normal DQN. At the episode level, the Rainbow agent won once every 2.0 episodes while the normal DQN won once every 4.0, meaning the DQN architecture stumbled its way to a win due to the maximum step is 3000 steps per episode. The Rainbow model consumed 5.17 million total steps; the normal DQN consumed 10.31 million. Higher win rate on half the data.

5.6.5 AAREN Exceeds LSTM Accuracy

The AAREN module achieved 24.23% piece identity inference accuracy, compared to 17.74% for the LSTM-based PBS module. The 6.5 percentage point gap confirms that attention-based history aggregation outperforms recurrent compression for this task. Section 5.1.4 (LSTM Accuracy Ceiling) predicted this limitation. The LSTM Q-values settled near 0.18 in late training. The Rainbow Q-values reached 0.84 by episode 9,500, indicating that the distributional representation maintains higher resolution across the value space.

5.6.6 Learned Exploration Active

The NoisyNet entropy rose steadily from 0.034 to 0.049 throughout Rainbow training, adapting to the need for exploration during the training as the agent have not converge to a solution. The normal DQN has a fixed entropy of zero (Section 5.1.5, Exploration Collapse), due to the constant decay of ϵ-greedy. Noisy Networks replaced frozen ϵ-greedy decay with adaptive, state-dependent exploration.

5.6.7 Extended Training at 75,000 Episodes

A separate Rainbow DQN with AAREN training run reached 75,047 episodes against heuristic opponents. This run used a different curriculum configuration and did not advance to self-play, so it cannot be directly compared to the self-play results above. It does, however, validate two architectural properties over a longer horizon. Table 5.4 summarizes its metrics.

Table 5.4: Rainbow DQN with AAREN extended training (75k episodes) against heuristic opponents.

Metric	**Value**
Total Episodes	75,047
Win Rate	11.66%
Draw Rate	77.02%
vs Smart Heuristic WR	11.74% (38,462 games)
vs Frozen Heuristic WR	11.59% (43,331 games)
Policy Loss (late)	2.84
Q-Value (late)	1.84
AAREN Accuracy	20.78%
Flag Capture Ratio	99.96%
Avg Episode Length	378

Loss Convergence Over Long Horizons

The distributional loss fell from 3.89 to 2.84 across 75,000 episodes, a clear downward trajectory that reinforces Section 5.6.3. The normal DQN showed no comparable long-term loss reduction; its loss either plateaued or inflated in late training. The near-zero correlation between the normal DQN's loss and win rate meant that additional training offered diminishing returns. The Rainbow architecture continued to extract value from each training episode, with the falling loss accompanied by a gradual increase in win rate across the second half of the run. Extended training works for the Rainbow agent because its loss function tracks a meaningful learning signal.

Flag Pursuit Behavior

The average episode length in the 75k run settled near 378 steps, compared to 1,345 for the normal DQN and 1,055 for the Rainbow self-play run. The shorter episodes indicate that the agent learned to end games faster. Average Q-values grew from near zero to 1.84, nearly five times the terminal normal DQN Q-value. This growth occurred alongside a 99.96% flag capture ratio among wins. The

Rainbow agent almost exclusively won by identifying and capturing the opponent flag rather than by attrition. The normal DQN won by flag capture only 18.79% of the time, relying instead on piece depletion. As training progressed, the agent shifted from passive survival toward purposeful flag pursuit, and the declining episode length reflects this shift. The attention mechanism enables this targeted strategic objective by allowing the agent to reason about the global board state and isolate the flag's probable location.

The 75k run also reveals an opponent-conditional effect. The win rate against heuristic opponents remained at 11.66% across 75,047 episodes, far below the 49.88% achieved in self-play at 9,778 episodes. This gap does not indicate that the Rainbow architecture fails against heuristics. Rather, it reflects the difference in what the agent can learn from each opponent class. Heuristic opponents follow fixed evaluation functions that do not adapt, so the agent cannot exploit predictable weaknesses in the way it can against a co-evolving self-play partner. The heuristic's consistent tactical play also denies the AAREN module the behavioral variation it needs to refine piece-identity inference. Against a self-play opponent whose policy improves alongside the agent, each game presents a different strategic landscape, driving both the policy and the belief model to generalize. The contrast shows that opponent diversity directly affects both convergence speed and the performance ceiling.

5.7 Evidence for Architectural Necessity

Each component in the thesis title maps to a specific failure mode observed in the normal DQN baseline and validated by the Rainbow DQN empirical results. Table 5.5 summarizes this mapping.

The combined findings support the full MARQ architecture. Each component targets a distinct, verified bottleneck. Draw dominance (Section 5.1.6) is a compound effect arising from the interaction of scalar value collapse, exploration death, and sparse reward propagation. No single enhancement can resolve it in isolation because each contributing failure mode persists independently. The empirical data confirms that the combined architectural response eliminated it (Section 5.6.2), reducing the draw rate from 50.04% to 6.69% while simultaneously doubling the win rate and restoring loss–performance coupling.

Table 5.5: MARQ title components mapped to normal DQN failure modes and Rainbow DQN validation.

Title Component	**Observed Failure Mode**	**Empirical Evidence**
Multi-Agent	Symmetric MARL Nash lock-in	P1 and P2 win rates converged to near-identical 25.1% and 24.9% in the normal DQN. The Rainbow agent broke this symmetry with a 49.88% win rate in self-play (Section 5.6.1).
Rainbow DQN	Q-value overestimation, scalar value collapse, exploration death, sparse reward propagation	Loss–performance coupling restored ($r = -0.787$, Section 5.6.3). Draw rate dropped from 50% to 6.69% (Section 5.6.2). NoisyNet entropy rose throughout training (Section 5.6.6).
AAREN	Sequential information bottleneck	AAREN accuracy 24.23% versus LSTM's 17.74% (Section 5.6.5). Q-value resolution improved from 0.18 to 0.84, enabling finer action discrimination.
Imperfect Information	Risk-averse draw-seeking policy	The 38.5% draw rate (Section 5.1.6) dropped to 6.69% (Section 5.6.2). Flag capture ratio rose to 99.96% over 75k episodes (Section 5.6.7), confirming the agent learned targeted flag pursuit under uncertainty.

5.8 Supporting Evidence from Related Work

These limitations are consistent with findings elsewhere in deep RL for imperfect information games.

Pérolat et al. (2022) solved full-size Stratego with DeepNash [35] using game-theoretic regularization (R-NaD), not Q-value maximization. DeepNash shows Stratego is solvable by deep RL, but only when the architecture explicitly handles non-stationarity and strategic diversity. The Ataraxos system [39] extended this to no-press Diplomacy with KL-divergence stabilization, a mechanism similar to MARQ's dynamic damping but absent from the normal DQN baseline.

Hessel et al. (2021) showed in the Rainbow DQN study that Double DQN, PER, Dueling Networks, Noisy Nets, C51, and multi-step returns each contribute additive gains [20]. Removing any single component degraded performance. No one fix is enough.

5.9 Implications for MARQ

The performance ceiling stems from identifiable architectural properties of the normal DQN, not from insufficient training or bad hyperparameters. The empirical validation confirms that the Rainbow DQN with AAREN addresses each limitation. Table 5.6 maps each finding to its theoretical cause, the MARQ component that addresses it, and its current validation status.

Table 5.6: Observed limitations mapped to theoretical causes, MARQ components, and validation status.

Finding	**Theoretical Cause**	**MARQ Component**	**Status**
Q-Value Overestimation	Maximization Bias	Double DQN / Dueling	Validated (Sec. 5.6.1)
Loss–Perf. Decoupling	Scalar Value Collapse	C51 Distributional RL	Validated (Sec. 5.6.3)
LSTM Accuracy Ceiling	Sequential Bottleneck	AAREN Attention	Validated (Sec. 5.6.5)
Exploration Collapse	Fixed ϵ-greedy	Noisy Networks	Validated (Sec. 5.6.6)
Reward Propagation	1-step Bootstrapping	n-step Returns	Validated (Sec. 5.6.4)
Draw Dominance	Compound Effect	Full MARQ Pipeline	Validated (Sec. 5.6.2)

The normal DQN baseline in this chapter served as a controlled reference point. The empirical data from three Rainbow DQN training runs validates each proposed enhancement. The Rainbow

agent doubled the win rate, eliminated draw dominance, restored loss–performance coupling, and demonstrated continued learning over 75,000 episodes.

Chapter 6

Conclusion and Recommendations

6.1 Conclusion

The experiments presented in this thesis set out to determine whether the performance ceiling of a normal DQN agent in Stratego arises from insufficient training or from structural properties of the architecture itself. The evidence supports the latter interpretation. Across 37,213 episodes of curriculum training, the normal DQN exhibited a stable 22% win rate, monotonically decreasing TD loss with no corresponding performance gain, Q-value overestimation, piece prediction accuracy frozen near 20%, and a 38.5% draw rate driven by passive piece shuffling. These observations are not independent failures. They form a coherent pattern: scalar value collapse prevents the agent from distinguishing aggressive high-variance moves from passive low-variance alternatives, exploration decay eliminates the mechanism for discovering new strategies, and the LSTM information bottleneck denies the agent the opponent model needed to evaluate combat outcomes.

The Rainbow DQN with AAREN addressed each of these bottlenecks through targeted architectural changes. C51 distributional value estimation replaced scalar Q-learning and restored the coupling between training loss and gameplay performance ($r = -0.787$ versus 0.051). Noisy Networks replaced frozen ϵ-greedy exploration with adaptive, state-dependent noise that continued expanding coverage throughout training. AAREN replaced the LSTM history encoder and raised piece-identity inference accuracy from 17.74% to 24.23%. Multi-step returns shortened the effective bootstrap depth for sparse terminal rewards. The combined effect was a win rate of 49.88% at 9,778

episodes, achieved on roughly half the environment steps consumed by the normal DQN, and a draw rate reduction from 50.04% to 6.69%.

The transition from draw-dominated outcomes to decisive games represents the most consequential behavioral shift observed in the data. The normal DQN converged on a policy that minimized short-term risk at the cost of long-term winning probability. The Rainbow agent, equipped with a distributional value head capable of representing multi-modal outcome distributions, learned to distinguish between moves that produce safe draws and moves that carry higher variance but lead to flag captures. The extended 75,047-episode training run confirmed that this aggressive posture matures into purposeful strategy: the agent achieved a 99.96% flag capture ratio among wins, with average episode length dropping to 378 steps. The agent did not merely learn to fight; it learned to identify and pursue the opponent flag.

These results connect each architectural component to a measured improvement over the baseline. Each component addresses a specific failure mode of the baseline, and the empirical gains match the theoretical predictions from distributional RL, attention-based sequence modeling, and learned exploration. The following sections present the individual component contributions, the broader contributions to reinforcement learning methodology, and the remaining limitations.

6.2 Recommended Architectural Enhancements

6.2.1 Value Estimation

The Q-value overestimation and scalar value collapse identified in the training analysis directly motivated two Rainbow DQN enhancements formalized in the MARQ architecture. The Dueling Network decomposes action values into state-value and advantage streams. This design suits Stratego's action space, where many positional moves share similar strategic value and only a small subset of actions (attacks, flag-adjacent moves) carry distinct utility. The C51 distributional head provides multi-modal value estimation that differentiates risky decisive actions from safe passive alternatives. Scalar Q-values provably cannot make this distinction.

The empirical results confirm this design. The correlation coefficient between loss and win rate shifted from 0.051 (normal DQN) to -0.787 (Rainbow DQN with AAREN), demonstrating that the distributional objective captures strategically meaningful value distinctions. The draw rate dropped from 50.04% to 6.69%, validating that multi-modal value estimation resolves the draw-seeking trap

caused by scalar value collapse.

6.2.2 LSTM to AAREN Transition

The LSTM prediction accuracy ceiling of approximately 18 to 22% was the most direct evidence for the AAREN module's necessity. The LSTM-based history encoder has concrete advantages over a full DRQN architecture in replay compatibility, per-position granularity, and gradient isolation. The sequential information bottleneck inherent in any recurrent encoder, however, limits representational capacity over long game horizons. AAREN preserves all structural advantages of the separated encoder design while replacing the recurrent computation with attention-based history aggregation. This substitution addresses the compression bottleneck without modifying the replay buffer infrastructure or the DQN backbone.

The empirical data validates this transition. AAREN achieved 24.23% piece identity inference accuracy compared to 17.74% for the LSTM, a 6.5 percentage point improvement. The separated encoder design itself contributes to the field. Independent hidden states for each of the 100 board positions produce a spatially structured $(64, 10, 10)$ embedding tensor that preserves piece-level temporal information. A monolithic DRQN hidden state would conflate action histories across all 40 opponent pieces into a single vector, losing spatial specificity. The per-position design also maintains full compatibility with standard experience replay by storing compact embedding snapshots alongside 15-channel board tensors, avoiding the sequence burn-in cost required by full DRQN architectures.

6.2.3 Exploration Recovery

The Noisy Net sigma remained at zero throughout the entire normal DQN training run. This confirmed that the ϵ-greedy exploration schedule exhausted its utility without the agent discovering strategies beyond the initial local optimum. Noisy Networks parameterize exploration as learnable per-weight perturbations and provide state-dependent exploration that adapts to the agent's uncertainty rather than following a predetermined decay schedule.

The Rainbow agent's NoisyNet entropy rose steadily from 0.034 to 0.049 throughout training, confirming that learned exploration noise continued to expand its coverage. This adaptive exploration contributed directly to the win rate improvement from 24.97% to 49.88%.

6.2.4 Long-Range Credit Assignment

Single-step TD learning attenuates terminal rewards by $\gamma^{200} \approx 0.134$ for mid-game decisions. Five-step returns ($n = 5$), as specified in MARQ, shorten the effective bootstrap depth and push strategic outcomes back to earlier decision points faster. The Rainbow agent's sample efficiency (1,060 steps per win versus 2,697 for the normal DQN) provides indirect evidence that multi-step returns accelerated reward propagation.

6.3 Computer Science Contributions

The following contributions apply to reinforcement learning systems beyond the Stratego domain. Stratego is the test domain, but the techniques are not domain-specific and apply to other partially observable or adversarial settings.

6.3.1 Five-Phase Curriculum Design

The curriculum transitions agents from full to partial observability in stages. Observability drops progressively while opponent difficulty rises and reward shaping decays. This mitigates the cold-start problem: agents do not need to learn basic mechanics and hidden-information reasoning at the same time. The structure applies wherever partial observability can be introduced gradually.

6.3.2 Gradient Bridge for Replay Compatibility

Hybrid architectures face a tension between replay accuracy and speed. Reconstructing full LSTM sequences at replay time gives correct gradients but costs 4× throughput. Detaching snapshot embeddings keeps throughput but kills the gradient path. The gradient bridge adds a differentiable epsilon-scaled zero-vector ($\epsilon \cdot W_{proj}(\mathbf{0})$) to stored embeddings, restoring the gradient path while keeping throughput high. The technique applies to any recurrent-encoder-plus-replay-buffer architecture.

6.3.3 Expectamax Search Integration

At test time, the system blends the DQN's positional Q-values with combat utilities derived from AAREN's rank-probability predictions through Expectamax search. This grounds neural value esti-

mates in tactical reality and blocks overconfident attacks on hidden pieces. The approach transfers to any partially observable domain that combines learned value functions with probabilistic state inference.

6.3.4 Potential-Based Reward Shaping with Phase Annealing

PBRS guarantees policy invariance while providing dense signals during early training. A multiplicative schedule tapers the shaping coefficient to zero across curriculum phases, so the final training phase runs on terminal rewards alone. No manual cutoff is needed. The design avoids the reward-farming problem common in additive shaping.

6.4 Limitations and Future Work

The validation presented in this thesis was bounded by the training infrastructure, opponent diversity, and compute budget available during the research period. The following limitations qualify the conclusions drawn above and identify directions where additional work is needed.

The Rainbow DQN with AAREN demonstrated strong empirical improvements over the normal DQN baseline, but the validation was conducted against heuristic and self-play opponents within the MARQ curriculum. Performance against opponents beyond the current heuristic and self-play pool may reveal gaps in strategic reasoning that the existing evaluation does not expose.

Combining C51, Noisy Nets, Dueling Networks, AAREN, and n-step returns in a multi-agent self-play loop introduces interaction effects that single-component theory does not predict. The extended 75k training run remained in the partial observability phase against heuristic opponents for its entire duration, suggesting that curriculum promotion thresholds require further tuning for long-horizon training stability.

The full MARQ pipeline is heavier than the normal DQN. Each checkpoint occupies approximately 433 MB compared to 39 MB, an 11-fold increase. While sample efficiency improved (5.17 million steps versus 10.31 million for the normal DQN), wall-clock time per episode is higher, reducing total episode count under a fixed compute budget.

Ablation studies are needed. The empirical data shows that the combined architecture outperforms the normal DQN, but confirmation that the gains are additive rather than redundant across individual components is still required. Future work should test each component in isolation against

the normal DQN baseline before assembling the full pipeline. The 75k extended training run provides a starting point for this analysis, as its long-horizon loss convergence and flag pursuit behavior suggest that the distributional and attention components contribute independently.

REFERENCES

[1] Adams, G., Padmanabhan, S., and Shekhar, S. Resolving implicit coordination in multi-agent deep rl with deep q-networks & game theory. *arXiv preprint arXiv:2012.09136* (2023).

[2] Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., Muneer, A., Sumiea, E. H., Alqushaibi, A., and Ragab, M. G. Rnn-lstm: From applications to modeling techniques and beyond—systematic review. *Journal of King Saud University - Computer and Information Sciences 36*, 5 (2024), 102068.

[3] Amaliani, R. N., Soewardikoen, D. W., Azhar, H., and Rahman, Y. Designing board games as an enhancer of a sense of community and cognition for the elderly of tresna werdha budi pertiwi social care. *Advances in Social Humanities Research 3*, 4 (2025).

[4] Author, F., and Author, S. Bridging local and global knowledge via transformer in board games. In *Proceedings of the International Joint Conference on Artificial Intelligence* (2025).

[5] Barzilai, G., Shamir, O., and Zamani, M. Are convex optimization curves convex? *arXiv preprint arXiv:2503.10138* (2025).

[6] Becker, T., and Sunberg, Z. Imperfect information games and counterfactual regret minimization in space domain awareness. In *Advanced Maui Optical and Space Surveillance Technologies Conference (AMOS)* (2022), AMOS Technologies.

[7] Bellemare, M. G., Dabney, W., and Rowland, M. *Distributional Reinforcement Learning*. MIT Press, 2023.

[8] Blüml, J., Czech, J., and Kersting, K. Alphaze**: Alphazero-like baselines for imperfect information games are surprisingly strong. *Frontiers in Artificial Intelligence Volume 6 - 2023* (2023).

[9] Brim, A. Deep reinforcement learning pairs trading with a double deep q-network. In *Proceedings of the 10th Annual Computing and Communication Workshop and Conference (CCWC)* (Las Vegas, Nevada, USA, Jan.–2020 2020), pp. 222–227.

[10] Brown, N., and Sandholm, T. Solving imperfect-information games via subgame solving and depth-limited search. *Artificial Intelligence 297* (2021), 103503.

[11] Canese, L., Cardarilli, G. C., Di Nunzio, L., Fazzolari, R., Giardino, D., Re, M., and Spanò, S. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences 11*, 11 (2021), 4948. Article number; MDPI journal uses article numbers instead of page ranges.

[12] Chen, S., Fried, D., and Topcu, U. Human-agent cooperation in games under incomplete information through natural language communication, 2024.

[13] Chitayat, P. P., Colman, A. M., Hyde, R. J., Drachen, A., and Cowling, P. Wards: Modelling the worth of vision in MOBA's. In *Intelligent Systems and Applications - Proceedings of the 2020 Intelligent Systems Conference (IntelliSys)* (2020), vol. 1251 of *Advances in Intelligent Systems and Computing*, Springer, pp. 55–76.

[14] Dbouk, H. M., Ghorayeb, K., Kassem, H., Hayek, H., Torrens, R., and Wells, O. Facility placement layout optimization. *Journal of Petroleum Science and Engineering 207* (2021), 109079.

[15] Devlin, S., and Kudenko, D. Potential-based reward shaping in large-scale mdps. *Reinforcement Learning Journal 3*, 2 (2022), 145–167.

[16] Echchahed, A., and Castro, P. S. A survey of state representation learning for deep reinforcement learning. *Transactions on Machine Learning Research* (2025).

[17] El Shar, I., and Jiang, D. Weakly coupled deep q-networks. In *Advances in Neural Information Processing Systems 36* (2023), Curran Associates, Inc. NeurIPS 2023 Conference Track.

[18] Feng, L., Tung, F., Hajimirsadeghi, H., Ahmed, M. O., Bengio, Y., and Mori, G. Attention as an rnn. *arXiv preprint arXiv:2405.13956* (2024).

[19] Harrington, J. Let's meetup? board game communities in hong kong. *Games and Culture 20*, 3 (2025), 279–297.

[20] Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. *Journal of Machine Learning Research 22*, 85 (2021), 1–52.

[21] Hu, C., Zhao, Y., Wang, Z., Du, H., and Liu, J. Games for artificial intelligence research: A review and perspectives. *IEEE Transactions on Artificial Intelligence 5*, 12 (2024), 5949–5968.

[22] Huang, Y. Deep q-networks. In *Deep Reinforcement Learning: Fundamentals, Research, and Applications*, S. Z. Hao Dong, Zihan Ding, Ed. Springer Nature, 2020, ch. 4, pp. 135–160. `http://www.deepreinforcementlearningbook.org`.

[23] Kaur, A., Kumar, K., Prakash, A., and Tripathi, R. Imperfect csi-based resource management in cognitive iot networks: A deep recurrent reinforcement learning framework. *IEEE Transactions on Cognitive Communications and Networking 9*, 5 (2023), 1271–1281.

[24] Kensert, A., Libin, P., Desmet, G., and Cabooter, D. Deep reinforcement learning for the direct optimization of gradient separations in liquid chromatography. *Journal of Chromatography A 1720* (2024), 464768.

[25] Le, N., Rathour, V. S., Yamazaki, K., Luu, K., and Savvides, M. Deep reinforcement learning in computer vision: A comprehensive survey, 2021.

[26] Li, B., Fang, Z., and Huang, L. Rl-cfr: Improving action abstraction for imperfect information extensive-form games with reinforcement learning. *arXiv preprint arXiv:2403.04344* (2024).

[27] Li, J., Zanuttini, B., and Ventos, V. Opponent-model search in games with incomplete information. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2024), vol. 38, pp. 9840–9847.

[28] Li, S., Zhang, Y., Wang, X., Xue, W., and An, B. Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)* (2021), IJCAI, pp. 3663–3669.

[29] Li, Z., Ji, Q., Ling, X., and Liu, Q. A comprehensive review of multi-agent reinforcement learning in video games. *arXiv preprint arXiv:2509.03682v1* (2025).

[30] Luo, Z., Chen, Z., and Welsh, J. Multi-agent reinforcement learning with deep networks for diverse q-vectors. *arXiv preprint arXiv:2406.07848v1* (2024). Version v1, June 2024.

[31] Nair, R. R., Babu, T., Kishore, S., Pavithra, K., and Sumana, S. G. Improving alpha-beta pruning efficiency in chess engines. In *2024 International Conference on IT Innovation and Knowledge Discovery (ITIKD)* (Manama, Bahrain, 2025), IEEE, pp. 1–6.

[32] Novikov, A., and Yanovskyi, V. Analysis of the decision-making algorithm efficiency in complex game environments on the example of pac-man. *Information Technologies and Computer Engineering 3*, 21 (2024), 108–118.

[33] Ouyang, X., and Zhou, T. Imperfect-information game ai agent based on reinforcement learning using tree search and a deep neural network. *Electronics 12*, 11 (2023), 2453.

[34] Petersen, C. The reality of the board game community: And the connection, identity and experience that creates it. Master's thesis, Eastern University, Aug. 2024.

[35] Pérolat, J., et al. Mastering the game of stratego with model-free multiagent reinforcement learning. *Science 378*, 6623 (2022), 990–996.

[36] Qu, T., Zhou, Q., Zhu, J., and Duan, F. Strategy optimization of imperfect information games based on nfsp with ddqn. In *Advances in Guidance, Navigation and Control*, L. Yan, H. Duan, and Y. Deng, Eds., vol. 845 of *Lecture Notes in Electrical Engineering*. Springer, Singapore, 2023, pp. 4376–4383.

[37] Saffidine, A., Finnsson, H., and Buro, M. Alpha-beta pruning for games with simultaneous moves. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021), vol. 26, pp. 556–562.

[38] Schmid, M., Moravčík, M., Burch, N., Kadlec, R., Davidson, J., Waugh, K., Bard, N., Timbers, F., Lanctot, M., Holland, G. Z., et al. Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances 9*, 46 (2023), eadg3256.

[39] Sokota, S., Vinitsky, E., Hu, H., Kolter, J. Z., and Farina, G. Superhuman ai for stratego using self-play reinforcement learning and test-time search, 2025.

[40] Sudhakar, A. V., Nekoei, H., Reymond, M., Liu, M., Rajendran, J., and Chandar, S. A generalist hanabi agent. *arXiv preprint arXiv:2503.14555v1* (2025).

[41] Wang, X., Li, Y., and Zhang, W. Deep reinforcement learning in imperfect-information games: A survey. *IEEE Transactions on Games 15*, 3 (2023), 421–438.

[42] Wang, Y. Game-theoretic understandings of multi-agent systems with multiple objectives, 2025.

[43] Wang, Y., Li, L., and Li, W. Constructing non-markovian decision process via history aggregator. *Applied Sciences 16*, 2 (2026), 955.

[44] Wang, Y., and Wellman, M. P. Regularization for strategy exploration in empirical game-theoretic analysis, 2023.

[45] Wongkamjan, W., Gu, F., Wang, Y., Hermjakob, U., May, J., Stewart, B., Kummerfeld, J., Peskoff, D., and Boyd-Graber, J. More victories, less cooperation: Assessing cicero's diplomacy play. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, p. 12423–12441.

[46] Xie, S., and Li, Z. Implicit bias of adamw: ℓ_∞ norm constrained optimization. *arXiv preprint arXiv:2404.04454v1* (2024).

[47] Xu, H., Li, K., Fu, H., Fu, Q., and Xing, J. Autocfr: Learning to design counterfactual regret minimization algorithms. In *Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-22)* (2022), Association for the Advancement of Artificial Intelligence.

[48] Xu, H., Li, K., Fu, H., FU, Q., Xing, J., and Cheng, J. Dynamic discounted counterfactual regret minimization. In *The Twelfth International Conference on Learning Representations* (2024).

[49] Zakharenkov, A., and Makarov, I. Deep reinforcement learning with dqn vs. ppo in vizdoom. In *2021 IEEE 21st International Symposium on Computational Intelligence and Informatics (CINTI)* (Budapest, Hungary, 2021), IEEE, pp. 131–136.

[50] Zhang, B., and Sandholm, T. Subgame solving without common knowledge. *Advances in Neural Information Processing Systems 34* (2021), 23993–24004.

[51] Zhang, B. H., and Sandholm, T. General search techniques without common knowledge for imperfect-information games, and application to superhuman fog of war chess. *arXiv preprint arXiv:2506.01242* (2025).

[52] Zhang, K., Yang, Z., and Başar, T. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of reinforcement learning and control* (2021), 321–384.

[53] Zheng, S., Trott, A., Srinivasa, S., Naik, N., Gruesbeck, M., Parkes, D. C., and Socher, R. The ai economist: Improving equality and productivity with ai-driven tax policies. *arXiv preprint arXiv:2004.13332* (2020).

[54] Zhu, K., Liu, Q., Yan, X., Wu, F., Zhao, X., Zhou, P., and Yang, X. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems 34*, 7 (2022), 3074–3093.